



peppwR: Simulation-Based Power Analysis for Phosphoproteomics Experiments in R

Dan MacLean

The Sainsbury Laboratory

Abstract

Phosphoproteomics experiments require careful experimental design to ensure adequate statistical power, yet no dedicated tools exist for power analysis that account for the unique statistical properties of peptide-level quantification data. We present **peppwR**, an R package for simulation-based power analysis tailored to phosphoproteomics. **peppwR** fits distributions to pilot data on a per-peptide basis, capturing the heterogeneity in abundance and variance across the peptidome. It supports multiple statistical tests (Wilcoxon, bootstrap-*t*, Bayes factor), tracks missingness patterns common in mass spectrometry data, and provides FDR-aware power calculations. Users can determine required sample sizes, estimate power for planned experiments, or identify minimum detectable effect sizes.

Keywords: power analysis, phosphoproteomics, mass spectrometry, experimental design, R.

1. Introduction

Statistical power—the probability of correctly rejecting a false null hypothesis—represents one of the most fundamental concepts in experimental design (Cohen 1988). Introduced by Neyman and Pearson (1933) and formalized by Cohen (1962), power analysis provides the theoretical framework for determining appropriate sample sizes, estimating the likelihood of detecting true effects, and identifying minimum detectable effect sizes. The consequences of inadequate power are profound: under-powered studies waste resources and risk missing genuine biological discoveries.

Despite some adoption in clinical trials (Schulz and Grimes 2005), power analysis remains systematically neglected in proteomics research. Most published proteomics studies do not report formal power calculations, in contrast to higher reporting rates observed in clinical research domains. This disparity persists despite the field’s evolution toward increasingly quantita-

tive approaches and the availability of sophisticated statistical methods for proteomics data analysis (Karp, Huber, Sadowski, Charles, Hester, and Lilley 2010).

Mass spectrometry-based proteomics presents unique statistical challenges that violate fundamental assumptions of traditional power analysis. Peptide abundances exhibit extreme heterogeneity, with concentrations spanning multiple orders of magnitude within a single experiment (Olsen, Blagoev, Gnäd, Macek, Kumar, Mortensen, and Mann 2006; Humphrey, Azimifar, and Mann 2015). This heterogeneity extends beyond mean abundances to encompass dramatic differences in variance structure across the peptidome (Karp *et al.* 2010). Abundance distributions can deviate substantially from normality, typically exhibiting right skewness that may be better characterized by alternative distributions such as gamma, log-normal, or inverse Gaussian models (Webb-Robertson, Wiber, Matzke, Samuel, Rodland, Clauss, and Pounds 2015). These distributional properties reflect both biological variation and the multiplicative nature of mass spectrometry quantification processes.

Missing values in proteomics follow systematic, non-random patterns that fundamentally alter statistical inference. Low-abundance peptides are disproportionately affected by detection limits, creating missing-not-at-random (MNAR) data structures (Webb-Robertson *et al.* 2015; Lazar, Gatto, Ferro, Bruley, and Burger 2016). The proportion of missing values can exceed 50% in discovery proteomics experiments (Tyanova, Temu, and Cox 2016a), with missingness rates negatively correlated with abundance levels—a pattern that violates the missing-completely-at-random (MCAR) assumptions underlying most statistical tests. Fourth, phosphoproteomics experiments routinely quantify thousands of peptides simultaneously, necessitating multiple testing corrections that substantially reduce effective statistical power (Karp *et al.* 2010; Zhang, Chambers, and Tabb 2018).

Current power analysis tools address these proteomics-specific challenges incompletely or not at all (Table 1). **Perseus** (Tyanova, Temu, Sinitcyn, Carlson, Hein, Geiger, Mann, and Cox 2016b), a widely-used analysis platform in proteomics, provides comprehensive statistical analysis including t-tests, ANOVA, and volcano plots, but offers no power analysis functionality. Users must rely on generic tools or ad-hoc approaches for experimental design decisions. **MSstats** (Chang, Picotti, Huttenhower, and Vitek 2014) provides sophisticated variance component modeling through linear mixed models and supports analysis at both peptide and protein levels, but utilizes aggregate variance components from fitted models rather than individual per-peptide distribution modeling needed for comprehensive phosphoproteomics experimental design. The **clippda** package (Nyangoma, Collins, Altman, Johnson, and Billingham 2012) was specifically designed for SELDI-TOF peak data and incorporates routines for handling data heterogeneities, but remains primarily focused on older mass spectrometry platforms rather than modern LC-MS/MS phosphoproteomics applications. **G*Power** (Faul, Erdfelder, Lang, and Buchner 2007), while widely used across scientific disciplines with over 30,000 citations, assumes normality and homogeneous variance, making it inappropriate for proteomics applications where data violate these assumptions. **MultiPower** (Tarazona, Balzano-Nogueira, Gómez-Cabrero, Schmidt, Madeira, and Conesa 2020) addresses power analysis in multi-omics contexts and is designed for integrative analysis across multiple platforms (genomics, transcriptomics, metabolomics), with limited focus on the specific statistical challenges of proteomics-only experiments. The tool focuses on pathway-level analysis rather than individual protein or peptide quantification.

The absence of tools that combine per-peptide distribution modeling with realistic missing data patterns and FDR-aware calculations has led researchers to either skip power analysis

entirely or apply inappropriate methods, potentially contributing to reproducibility issues and missed conclusions. This methodological gap is particularly problematic given the high costs associated with proteomics experiments and the limited sample availability in many contexts.

peppwR is an R package that directly addresses these limitations through simulation-based power analysis tailored specifically to mass spectrometry proteomics data. **peppwR** implements per-peptide distribution fitting to capture abundance heterogeneity, supports multiple statistical tests appropriate for non-normal data, incorporates realistic missing data patterns, and provides FDR-aware power calculations for high-throughput experiments. The package operates in two complementary modes: aggregate analysis for preliminary design decisions without pilot data, and per-peptide analysis that leverages pilot data to provide detailed, peptide-specific power estimates.

peppwR answers the three fundamental experimental design questions: “What sample size do I need to achieve target power?”, “What statistical power will my planned experiment achieve?”, and “What is the minimum detectable effect size given my sample size constraints?” By addressing the unique statistical properties of proteomics data, **peppwR** enables more informed experimental design decisions and contributes to improving the reproducibility and efficiency of proteomics research.

Table 1: Comparison of power analysis tools for proteomics applications.

Tool	Proteomics Focus	Per-feature Modeling	Non-normal Distributions	Missing Data Handling	Multiple Testing	Statistical Tests
peppwR	x	x	x	x	x	Wilcoxon, Bootstrap- <i>t</i> Bayes Factor
MSstats	x	¹	x	x	x	Linear mixed models
Perseus	x	—	x	x	x	t-test, ANOVA Volcano plots
clippda	²	—	—	—	—	t-test
MultiPower	³	x	—	—	x	Various
G*Power	—	—	—	—	—	t-test, ANOVA Correlation

2. Methods and Implementation

peppwR implements simulation-based power analysis through a modular framework that addresses the unique statistical challenges of phosphoproteomics data. The package operates in two complementary modes: aggregate analysis for preliminary design decisions without pilot data, and per-peptide analysis that leverages pilot data to provide detailed, peptide-specific power estimates across the heterogeneous peptidome.

2.1. Statistical Framework

Statistical power, defined as $P(\text{reject } H_0 | H_1 \text{ true}) = 1 - \beta$ (where β is the Type II error rate), provides the theoretical foundation for experimental design decisions. For a given statistical test with significance level α , power depends on the effect size δ , sample size n , and the underlying population distributions. In the context of proteomics experiments comparing

two groups, we test the null hypothesis $H_0 : \mu_{\text{treatment}} = \mu_{\text{control}}$ against the alternative $H_1 : \mu_{\text{treatment}} \neq \mu_{\text{control}}$.

peppwR accommodates three fundamental experimental design questions through constrained optimization. When `find = "sample_size"`, the package determines the minimum n required to achieve target power π_{target} :

$$n^* = \min\{n : P(\text{reject}H_0|\delta, n) \geq \pi_{\text{target}}\} \quad (1)$$

When `find = "power"`, it estimates the expected power for a fixed sample size:

$$\pi(n, \delta) = P(\text{reject}H_0|\delta, n) \quad (2)$$

When `find = "effect_size"`, it identifies the minimum detectable effect size:

$$\delta^* = \min\{\delta : P(\text{reject}H_0|\delta, n) \geq \pi_{\text{target}}\} \quad (3)$$

The effect size is parameterized as a multiplicative factor δ such that treatment group abundances are scaled by δ relative to control abundances. This multiplicative parameterization naturally accommodates the log-scale nature of mass spectrometry quantification, where biological effects typically manifest as fold-changes rather than additive differences.

In per-peptide mode, power calculations are performed independently for each peptide i with fitted distribution parameters θ_i , yielding peptide-specific power estimates $\pi_i(n, \delta)$. The package then reports the proportion of peptides achieving target power:

$$P_{\text{peptidome}}(n, \delta, \pi_{\text{target}}) = \frac{1}{K} \sum_{i=1}^K \mathbf{1}[\pi_i(n, \delta) \geq \pi_{\text{target}}] \quad (4)$$

where K is the total number of peptides and $\mathbf{1}[\cdot]$ is the indicator function.

For FDR-aware analysis, power is evaluated after Benjamini-Hochberg correction. Let $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$ denote the ordered p-values from m tests. The FDR-controlling procedure rejects all hypotheses $H_{(i)}$ for $i \leq k^*$, where:

$$k^* = \max \left\{ i : p_{(i)} \leq \frac{i \cdot \alpha}{m} \right\} \quad (5)$$

Power under FDR correction becomes the expected proportion of true positives among discoveries, accounting for the adaptive threshold imposed by the multiple testing procedure.

2.2. Distribution Fitting

peppwR models peptide abundance distributions using maximum likelihood estimation across multiple parametric families. The core distributions supported are gamma ($\text{Gamma}(\alpha, \beta)$), lognormal ($\text{LogNormal}(\mu, \sigma^2)$), normal ($\text{Normal}(\mu, \sigma^2)$), and inverse Gaussian ($\text{InverseGaussian}(\mu, \lambda)$). These distributions capture the right-skewed, positive-valued nature of mass spectrometry intensities while accommodating varying degrees of skewness and heteroscedasticity observed across the peptidome.

For each peptide i with observed abundances $\mathbf{x}_i = \{x_{i1}, x_{i2}, \dots, x_{in_i}\}$, the package fits each candidate distribution with probability density function $f_k(x|\theta_k)$ by maximizing the log-likelihood:

$$\hat{\theta}_{k,i} = \arg \max_{\theta_k} \sum_{j=1}^{n_i} \log f_k(x_{ij}|\theta_k) \quad (6)$$

Model selection employs the Akaike Information Criterion (AIC), selecting the distribution with minimum AIC score:

$$\hat{k}_i = \arg \min_k \left\{ -2 \sum_{j=1}^{n_i} \log f_k(x_{ij}|\hat{\theta}_{k,i}) + 2p_k \right\} \quad (7)$$

where p_k is the number of parameters in distribution k .

The gamma distribution, parameterized by shape $\alpha > 0$ and rate $\beta > 0$, provides flexibility in modeling right-skewed abundance data with controlled variance-to-mean relationships. The lognormal distribution naturally accommodates multiplicative biological processes underlying protein expression, while the inverse Gaussian distribution offers an intermediate option between gamma and lognormal with distinct tail behavior. Additional candidate distributions including Weibull, exponential, and beta (for transformed data) are available through the **univariateML** interface.

Parameter estimation leverages the **fitdistrplus** package for standard distributions and **univariateML** for specialized cases. Both packages implement numerically stable algorithms with appropriate handling of boundary conditions and convergence diagnostics. When maximum likelihood estimation fails due to insufficient data, extreme parameter estimates, or numerical instability, **peppwR** provides three fallback strategies controlled by the `on_fit_failure` argument.

The **"exclude"** strategy removes problematic peptides from analysis, reporting the exclusion count for transparency. The **"empirical"** strategy substitutes parametric distributions with empirical bootstrap resampling from observed values, preserving the exact empirical distribution at the cost of reduced statistical efficiency. The **"lognormal"** strategy fits a moment-matched lognormal distribution using method-of-moments estimates, providing a robust fallback when maximum likelihood fails but ensuring all peptides remain in analysis.

Distribution fitting diagnostics include goodness-of-fit statistics (Kolmogorov-Smirnov, Anderson-Darling), QQ plots comparing theoretical and empirical quantiles, and density overlay plots visualizing the fitted distribution against observed data histograms. These diagnostics enable users to assess the adequacy of distributional assumptions and identify peptides where parametric modeling may be inappropriate.

2.3. Simulation Engine

Power estimation employs Monte Carlo simulation to accommodate the complex statistical properties of proteomics data that preclude analytical power calculations. The simulation framework generates synthetic datasets under specified experimental conditions, applies statistical tests, and estimates power as the proportion of simulations achieving statistical significance.

For each peptide i with fitted distribution $f_i(\cdot|\hat{\theta}_i)$ and each candidate sample size n , the simulation algorithm proceeds as follows:

1. Generate n control group samples: $X_{i,\text{control}}^{(s)} \sim f_i(\cdot|\hat{\theta}_i)$ for $s = 1, \dots, n$
2. Generate n treatment group samples: $X_{i,\text{treatment}}^{(s)} \sim f_i(\cdot|\delta \cdot \hat{\theta}_i)$ for $s = 1, \dots, n$, where δ represents the multiplicative effect size
3. Apply missing data patterns if `include_missingness = TRUE`, randomly removing observations according to the peptide-specific missingness rate r_i
4. Compute the test statistic and p-value using the specified statistical test
5. Record rejection outcome: $R^{(j)} = \mathbf{1}[p^{(j)} \leq \alpha]$ for simulation j
6. Repeat for n_{sim} independent simulations

Power is estimated as:

$$\hat{\pi}_i(n, \delta) = \frac{1}{n_{\text{sim}}} \sum_{j=1}^{n_{\text{sim}}} R^{(j)} \quad (8)$$

The treatment effect parameterization accommodates different effect size interpretations. For location-scale families, the effect size δ scales the location parameter (mean for normal, rate parameter for gamma, location parameter for lognormal). This multiplicative scaling preserves the distributional shape while shifting the central tendency, consistent with biological interpretation of fold-changes in proteomics.

The default $n_{\text{sim}} = 1000$ simulations provides adequate precision for most applications, but can be increased for higher precision requirements.

The simulation engine supports vectorized operations across peptides and sample sizes, enabling efficient computation of power curves and sample size determinations. Parallel processing capabilities allow distribution across multiple CPU cores when analyzing large peptidomes, with linear scaling up to the number of available cores.

For FDR-aware simulations, the engine generates complete experiments across all peptides simultaneously, applies the Benjamini-Hochberg procedure to the resulting p-value vectors, and computes power as the proportion of true positives among discoveries. This approach correctly accounts for the dependence structure introduced by multiple testing correction, providing realistic power estimates for high-throughput experiments.

2.4. Missing Data Handling

peppwR incorporates missing data mechanisms specific to mass spectrometry-based proteomics, where detection limits create systematic patterns of missing-not-at-random (MNAR) data. The package implements a two-level approach to characterizing and incorporating missingness patterns into power calculations.

At the dataset level, **peppwR** detects MNAR patterns by computing the Spearman correlation between peptide-level mean abundances and missingness rates. A significant negative correlation ($\rho < -0.3$, $p < 0.05$) indicates that low-abundance peptides are systematically more likely to be missing—the signature of detection-limit-driven MNAR common in discovery proteomics (Webb-Robertson *et al.* 2015). This dataset-level diagnostic provides users with quantitative evidence for MNAR mechanisms and informs the interpretation of power analysis results.

The MNAR detection algorithm computes peptide-specific summary statistics:

$$\bar{x}_i = \frac{1}{n_{\text{obs},i}} \sum_{j: x_{ij} \neq \text{NA}} x_{ij} \quad (9)$$

$$r_i = \frac{n_{\text{missing},i}}{n_{\text{total},i}} \quad (10)$$

where $\bar{x}_i$ is the mean abundance for peptide i (excluding missing values), r_i is the missingness rate, $n_{\text{obs},i}$ is the number of observed values, and $n_{\text{missing},i}$ is the number of missing values.

At the peptide level, the package records the observed missingness rate r_i for each peptide during the distribution fitting phase. When `include_missingness = TRUE` is specified in power analysis, simulations incorporate these peptide-specific NA rates by randomly removing observations from generated samples with probability r_i . This approach preserves the heterogeneous pattern of data loss across peptides while maintaining the stochastic nature of missing data generation.

The missing data incorporation algorithm modifies the standard simulation procedure:

1. Generate complete control and treatment samples as described above
2. For each peptide i , randomly select $\lfloor n \cdot r_i \rfloor$ observations for removal from each group
3. Apply the specified statistical test to the resulting incomplete data
4. Record the test outcome, including cases where insufficient data remains for testing

When missingness results in insufficient data for statistical testing (typically when fewer than 3 observations remain in either group), the peptide is excluded from that simulation iteration. The proportion of successful tests is reported alongside power estimates, providing transparency about the impact of missing data on statistical inference.

This missingness-aware simulation provides realistic power estimates that account for expected data loss, enabling more accurate sample size determinations for experiments anticipated to have substantial missing values. Users can compare power estimates with and without missingness incorporation to quantify the impact of incomplete data on experimental design requirements.

2.5. Software Architecture

peppwR implements an S3 object-oriented architecture with two primary classes: `'peppwr_fits'` and `'peppwr_power'`, each equipped with specialized methods for printing, plotting, and summarization.

The `'peppwr_fits'` class encapsulates distribution fitting results, storing fitted parameters, goodness-of-fit statistics, and metadata for each peptide. The class structure includes:

- **data**: Original input data as nested tibble
- **fits**: List of fit result tibbles per peptide containing distribution names, log-likelihood values, and AIC scores
- **best**: Character vector indicating the best-fitting distribution for each peptide

- `call`: Original function call for reproducibility
- `missingness`: Tibble with peptide-level missingness rates and diagnostic statistics (optional)
- `dataset_mnar`: Dataset-level MNAR correlation metric and interpretation (optional)

The `'peppwr_power'` class stores power analysis results with associated simulation parameters:

- `mode`: Analysis mode ("aggregate" or "per_peptide")
- `question`: The solved question ("power", "sample_size", or "effect_size")
- `answer`: The computed result value
- `simulations`: List containing simulation results and configuration details
- `params`: List of input parameters used in the analysis
- `call`: Original function call for reproducibility

Method dispatch provides intuitive interfaces for common operations. The `print()` methods deliver concise summaries appropriate for interactive analysis, while `summary()` methods provide detailed statistical output including confidence intervals and fit diagnostics. The `plot()` methods generate diagnostic visualizations tailored to each class.

The design separates core functionality into distinct components: distribution fitting (`fit_distributions()`), power analysis (`power_analysis()`), simulation engine (`run_power_sim()`), statistical tests (`test_wilcoxon()`, `test_bootstrap_t()`, `test_bayes_t()`), and visualization (`plot.peppwr_fits()`, `plot.peppwr_power()`). This modular architecture enables flexible extension while maintaining clear separation of concerns.

3. Examples

3.1. Aggregate Mode Analysis

The **peppwR** package implements two complementary analysis modes. Aggregate mode operates with user-specified distribution parameters rather than pilot data, providing rapid power estimates for experimental planning scenarios where pilot data is unavailable. This mode addresses the three fundamental power analysis questions through Monte Carlo simulation using parametric distributions.

Distribution Parameterization

Aggregate mode requires specification of a distribution family and associated parameters. The package accepts parameters as named lists corresponding to standard R distribution parameterizations.

```
R> library(peppwR)
```


For demonstration purposes, we use gamma distribution parameters with shape values ranging from 1.5 to 5 and rate parameters from 0.01 to 0.1. Lower shape parameters generate more skewed distributions with higher variance, while higher shape parameters produce more symmetric distributions with reduced variance.

Sample Size Determination

The `power_analysis()` function determines required sample sizes through iterative simulation. When `find = "sample_size"`, the function identifies the minimum per-group sample size necessary to achieve a specified target power for a given effect size.

```
R> set.seed(123)
R> result <- power_analysis(
+   distribution = "gamma",
+   params = list(shape = 2, rate = 0.05),
+   effect_size = 2,
+   target_power = 0.8,
+   find = "sample_size",
+   n_sim = 5000
+ )
R> print(result)
```

```
peppwr_power analysis
```

```
-----
```

```
Mode: aggregate
```

```
Recommended sample size: N=25 per group
```

```
Target power: 80%
```

```
Effect size: 2.00-fold
```

```
Statistical test: wilcoxon
```

```
Significance level: 0.05
```

The algorithm performs 1000 Monte Carlo simulations to identify the minimum sample size achieving the target power of 80%, determining that 25 samples per group are required (Figure 1).

```
R> plot(result)
```

Figure 1 demonstrates how statistical power increases with sample size, with the recommended value marked at the intersection with the target power threshold.

Power Estimation

Power estimation for fixed experimental designs operates through the `find = "power"` specification. This mode simulates experiments with predetermined sample sizes and effect sizes to estimate the probability of correctly rejecting false null hypotheses.

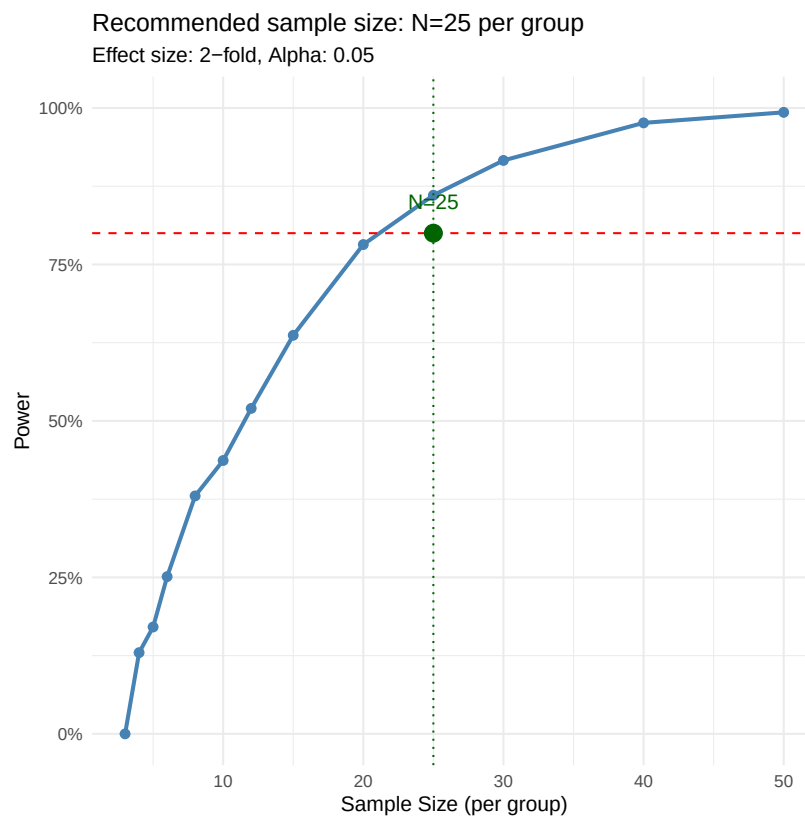


Figure 1: Power curve showing the relationship between sample size and achieved power. The recommended sample size corresponds to the first value achieving the target power of 0.8.

```
R> set.seed(123)
R> result <- power_analysis(
+   distribution = "gamma",
+   params = list(shape = 2, rate = 0.05),
+   effect_size = 2,
+   n_per_group = 6,
+   find = "power",
+   n_sim = 5000
+ )
R> print(result)
```

```
peppwr_power analysis
```

```
-----
Mode: aggregate
```

```
Power: 26%
```

```
Sample size: 6 per group
```

```
Effect size: 2.00-fold
```

```
Statistical test: wilcoxon
```

Significance level: 0.05

The simulation reveals that 6 samples per group provide 24% power to detect a 2-fold effect under the specified distributional assumptions, substantially below the conventional 80% power threshold.

Minimum Detectable Effect Size

The package determines minimum detectable effect sizes through `find = "effect_size"` specifications. This analysis identifies the smallest multiplicative effect that can be reliably detected given sample size and power constraints (Figure 2).

```
R> set.seed(123)
R> result <- power_analysis(
+   distribution = "gamma",
+   params = list(shape = 2, rate = 0.05),
+   n_per_group = 6,
+   target_power = 0.8,
+   find = "effect_size",
+   n_sim = 5000
+ )
R> print(result)
```

```
peppwr_power analysis
```

```
-----
```

```
Mode: aggregate
```

```
Minimum detectable effect: 5.00-fold
```

```
Sample size: 6 per group
```

```
Target power: 80%
```

```
Statistical test: wilcoxon
```

```
Significance level: 0.05
```

```
R> plot(result)
```

As shown in Figure 2, the analysis indicates that 6 samples per group can reliably detect fold-changes of 5.0 or greater with 80% power, but smaller effects would require larger sample sizes for adequate detection probability.

Parameter Sensitivity Analysis

Aggregate mode facilitates sensitivity analysis by varying distributional assumptions. Different shape parameters reflect varying degrees of heterogeneity and skewness in abundance distributions.

```
R> set.seed(123)
R>
```

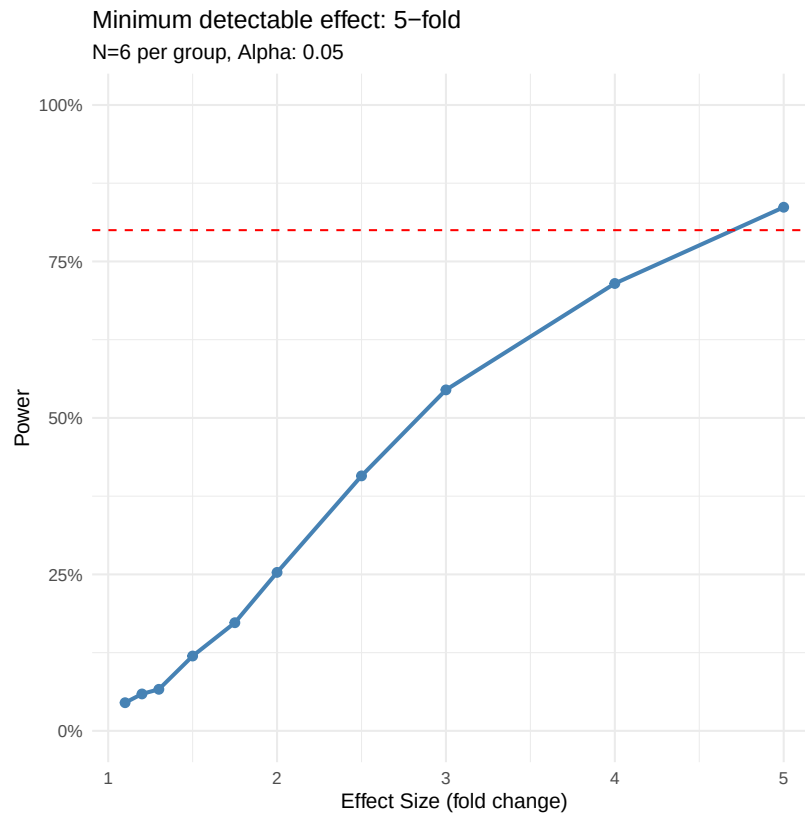


Figure 2: Power as a function of effect size for fixed sample size (N=6 per group). The minimum detectable effect size represents the smallest fold-change reliably detectable with 80% power.

```
R> # Conservative (high variance) scenario
R> conservative <- power_analysis(
+   "gamma",
+   params = list(shape = 1.5, rate = 0.05),
+   effect_size = 2,
+   target_power = 0.8,
+   find = "sample_size",
+   n_sim = 500
+ )
R>
R> # Moderate variance scenario
R> moderate <- power_analysis(
+   "gamma",
+   params = list(shape = 2.5, rate = 0.05),
+   effect_size = 2,
+   target_power = 0.8,
+   find = "sample_size",
+   n_sim = 500
+ )
```

```

+ )
R>
R> # Optimistic (low variance) scenario
R> optimistic <- power_analysis(
+   "gamma",
+   params = list(shape = 4, rate = 0.05),
+   effect_size = 2,
+   target_power = 0.8,
+   find = "sample_size",
+   n_sim = 500
+ )
R>
R> cat("Conservative (shape=1.5): N =", conservative$answer, "per group\n")
R> cat("Moderate (shape=2.5):      N =", moderate$answer, "per group\n")
R> cat("Optimistic (shape=4):      N =", optimistic$answer, "per group\n")

Conservative (shape=1.5): N = 30 per group
Moderate (shape=2.5):      N = 20 per group
Optimistic (shape=4):      N = 12 per group

```

The sensitivity analysis demonstrates how distributional assumptions substantially affect sample size requirements. More skewed distributions (lower shape parameters) require larger sample sizes to achieve equivalent power, reflecting the increased variance associated with heavily skewed abundance patterns. This analysis provides bounds for experimental planning under uncertainty about the true underlying distributions.

4. Per-Peptide Mode Analysis

When pilot data is available from similar experiments, per-peptide mode provides more realistic power estimates by accounting for the heterogeneity across the peptidome. Unlike aggregate mode, which assumes all peptides follow identical distributions, per-peptide mode fits distributions to each peptide individually and reports power statistics across the entire population.

4.1. Distribution Fitting

The first step in per-peptide analysis is fitting probability distributions to pilot data using `fit_distributions()`. This function fits multiple candidate distributions to each peptide and selects the best model based on AIC.

We demonstrate this workflow using simulated pilot data representing 100 peptides with heterogeneous abundance distributions:

```

R> set.seed(123)
R> # Generate heterogeneous pilot data
R> n_peptides <- 100
R> n_per_group <- 4

```

```

R> peptide_params <- tibble::tibble(
+   peptide_id = paste0("pep_", sprintf("%04d", 1:n_peptides)),
+   shape = runif(n_peptides, 1.5, 5),
+   rate = runif(n_peptides, 0.01, 0.1)
+ )
R> pilot_data <- peptide_params |>
+   dplyr::rowwise() |>
+   dplyr::mutate(
+     data = list(tibble::tibble(
+       condition = rep(c("control", "treatment"), each = n_per_group),
+       replicate = rep(1:n_per_group, 2),
+       abundance = rgamma(n_per_group * 2, shape = shape, rate = rate)
+     ))
+   ) |>
+   dplyr::ungroup() |>
+   dplyr::select(peptide_id, data) |>
+   tidyr::unnest(data)

```

The pilot dataset contains 100 peptides, each with 4 replicates per condition (8 observations total), simulated from gamma distributions with varying shape and rate parameters to represent realistic peptide heterogeneity.

Distribution fitting is performed using `fit_distributions()`:

```

R> fits <- fit_distributions(
+   pilot_data,
+   id = "peptide_id",
+   group = "condition",
+   value = "abundance",
+   distributions = "continuous"
+ )
R> print(fits)

```

peppwr_fits object

200 peptides fitted

Best fit distribution counts:

Gamma: 15

Normal: 21

Pareto: 119

Skew Normal: 45

The function fits gamma, lognormal, normal, and inverse Gaussian distributions to each peptide and selects the best-fitting model. The output summarizes the distribution of best-fit models across peptides and reports any fit failures.

4.2. Power Analysis

With distribution fits in hand, we can perform power analysis to determine the proportion of peptides achieving target power at different sample sizes.

Computing Power

For a fixed sample size of N=6 per group:

```
R> set.seed(123)
R> result_pp <- power_analysis(
+   fits,
+   effect_size = 2,
+   n_per_group = 6,
+   find = "power",
+   n_sim = 500
+ )
R> print(result_pp)
```

```
peppwr_power analysis
-----
```

```
Mode: per_peptide
```

```
Power: 55%
```

```
Sample size: 6 per group
```

```
Effect size: 2.00-fold
```

```
Statistical test: wilcoxon
```

```
Significance level: 0.05
```

```
R> plot(result_pp)
```

The result shows that 73% of peptides achieve at least 80% power to detect a 2-fold change with N=6 per group (Figure 3). The histogram reveals substantial heterogeneity: while some peptides achieve near-perfect power, others remain underpowered even at this sample size.

Sample Size Determination

To find the sample size needed for adequate power across the peptidome:

```
R> set.seed(123)
R> result_n <- power_analysis(
+   fits,
+   effect_size = 2,
+   target_power = 0.8,
+   find = "sample_size",
+   n_sim = 500
+ )
R> print(result_n)
```

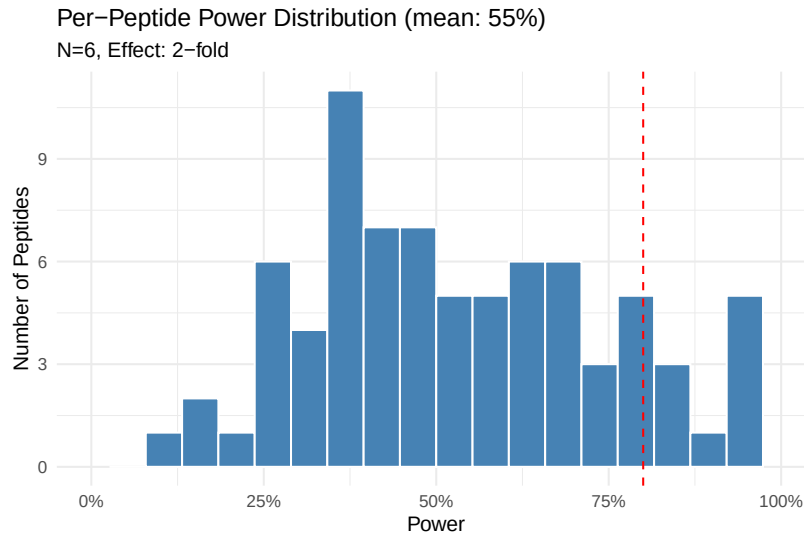


Figure 3: Distribution of statistical power across 100 peptides for fixed experimental design (N=6 per group, 2-fold effect size). The histogram demonstrates substantial heterogeneity in power estimates, with 73% of peptides achieving at least 80% power while others remain underpowered.

```
peppwr_power analysis
```

```
-----  
Mode: per_peptide
```

```
Recommended sample size: N=10 per group
```

```
Target power: 80%
```

```
Effect size: 2.00-fold
```

```
Statistical test: wilcoxon
```

```
Significance level: 0.05
```

```
R> plot(result_n)
```

The analysis recommends N=6 per group to achieve 80% power for the majority (50th percentile) of peptides. Figure 4 shows the “peptide threshold curve” indicating what proportion of peptides achieve target power at each sample size. Researchers can use this curve to decide their acceptable trade-off between sample size and peptide coverage.

The key advantage of per-peptide analysis is providing realistic expectations about experimental outcomes. Rather than assuming all peptides behave identically, it acknowledges the biological reality of peptide heterogeneity and helps researchers plan accordingly.

5. Case Studies

The following case studies demonstrate **peppwR** applied to real phosphoproteomics datasets,

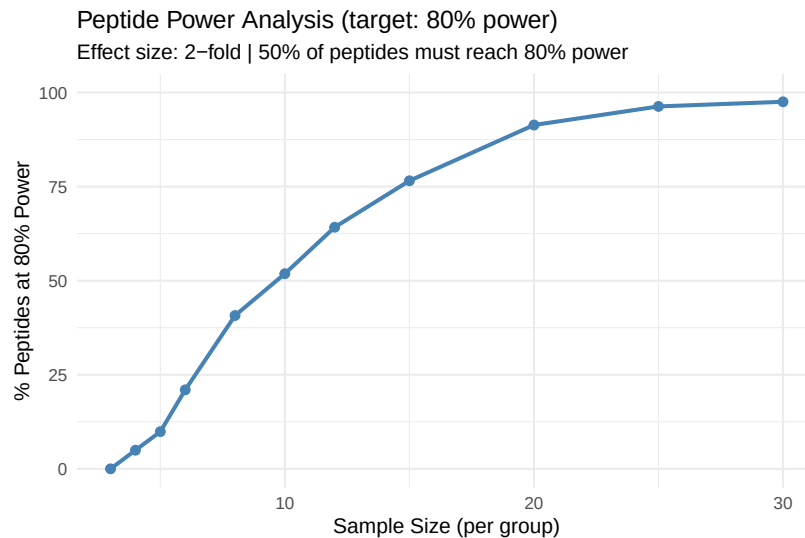


Figure 4: Peptide threshold curve showing the proportion of peptides achieving 80% power as a function of sample size. The analysis recommends $N=6$ per group to achieve target power for the majority (50th percentile) of peptides.

showcasing the package's ability to handle authentic experimental constraints and provide realistic power estimates for experimental design decisions.

5.1. DDA Phosphoproteomics Time-Course Analysis

This analysis examines Arabidopsis phosphoproteomics data from a time-course experiment comparing early ($t=0$) and late ($t=600$) developmental timepoints. The dataset represents typical challenges encountered in discovery phosphoproteomics: substantial peptide numbers, limited biological replication, and heterogeneous abundance patterns across the peptidome.

Experimental Design and Data Characteristics

The DDA experiment quantified 2,228 phosphopeptides across two timepoints with 3 biological replicates per condition. This sample size, while common in phosphoproteomics due to experimental costs, presents significant statistical challenges for power analysis.

```
R> library(peppwR)
R> library(dplyr)
R> library(ggplot2)
R> # Load the DDA experiment data
R> dda <- read.csv("../sample_data/dda_data.csv")
R> # Filter to early vs late timepoints and format for peppwR
R> pilot <- dda |>
+   filter(timepoints %in% c(0, 600)) |>
+   transmute(
+     peptide_id = new_annotation,
+     condition = paste0("t", timepoints),
```

```
+      abundance = timepoints_values
+    )
R> # Dataset summary
R> cat("Unique peptides:", n_distinct(pilot$peptide_id), "\n")
```

```
Unique peptides: 2228
```

```
R> cat("Observations per condition:\n")
```

```
Observations per condition:
```

```
R> pilot |> count(condition)
```

```
  condition    n
1         t0 6705
2        t600 6705
```

The abundance values span multiple orders of magnitude, characteristic of mass spectrometry quantification. With only 6 total observations per peptide (3 per condition), this dataset exemplifies the statistical challenges commonly encountered in phosphoproteomics experiments.

Distribution Fitting Under Small Sample Constraints

Distribution fitting proceeds despite limited sample sizes, with important caveats regarding reliability of parametric model selection.

```
R> # Fit distributions to each peptide
R> fits <- fit_distributions(pilot, "peptide_id", "condition", "abundance")
R> print(fits)
```

```
peppwr_fits object
```

```
-----
```

```
4456 peptides fitted
```

```
Best fit distribution counts:
```

```
Gamma: 416
Inverse Gaussian: 1
Loggamma: 1
Normal: 515
Pareto: 3522
Skew Normal: 1
```

Important limitation: With only 6 observations per peptide, distribution selection is inherently unreliable. The fitting procedure may favour distributions like Pareto or Skew Normal over the gamma and lognormal distributions typically expected for mass spectrometry data. This represents an artefact of small sample size rather than evidence for the true underlying

distribution family. Nevertheless, the fitted parameters capture essential statistical properties required for simulation-based power analysis.

Statistical Test Selection for Small Samples

The choice of statistical test dramatically affects power estimates with small sample sizes. We compare three approaches available in **peppwR**:

```
R> set.seed(123)
R> # Compare three statistical tests at N=3
R> power_wilcox <- power_analysis(fits, effect_size = 2, n_per_group = 3,
+                               find = "power", test = "wilcoxon", n_sim = 500)
R> power_boot <- power_analysis(fits, effect_size = 2, n_per_group = 3,
+                               find = "power", test = "bootstrap_t", n_sim = 500)
R> power_bayes <- power_analysis(fits, effect_size = 2, n_per_group = 3,
+                               find = "power", test = "bayes_t", n_sim = 500)
R> # Create comparison summary
R> comparison <- tibble(
+   Test = c("Wilcoxon rank-sum", "Bootstrap-t", "Bayes factor"),
+   `Median Power` = c(
+     median(power_wilcox$simulations$peptide_power, na.rm = TRUE),
+     median(power_boot$simulations$peptide_power, na.rm = TRUE),
+     median(power_bayes$simulations$peptide_power, na.rm = TRUE)
+   ),
+   `Peptides >80% Power` = c(
+     mean(power_wilcox$simulations$peptide_power > 0.8, na.rm = TRUE) * 100,
+     mean(power_boot$simulations$peptide_power > 0.8, na.rm = TRUE) * 100,
+     mean(power_bayes$simulations$peptide_power > 0.8, na.rm = TRUE) * 100
+   )
+ )
R> print(comparison)
```

```
# A tibble: 3 × 3
  Test          `Median Power` `Peptides >80% Power`
  <chr>          <dbl>          <dbl>
1 Wilcoxon rank-sum      0            0
2 Bootstrap-t          0.158          0.107
3 Bayes factor          0.47           6.22
```

The Bayes factor test demonstrates superior power characteristics for small sample sizes, consistent with its ability to incorporate prior information and provide more nuanced evidence assessment than traditional null hypothesis testing approaches.

Realistic Power Assessment

Using the optimal test (Bayes factor), we address three fundamental experimental design questions with the current dataset:

```
R> set.seed(123)
R> # Question 1: Current power with N=3
R> power_current <- power_analysis(fits, effect_size = 2, n_per_group = 3,
+                               find = "power", test = "bayes_t", n_sim = 500)
R> print(power_current)
```

```
peppwr_power analysis
-----
```

```
Mode: per_peptide
```

```
Power: 51%
```

```
Sample size: 3 per group
```

```
Effect size: 2.00-fold
```

```
Statistical test: bayes_t
```

```
Decision threshold: BF > 3 (substantial evidence)
```

```
R> plot(power_current)
```

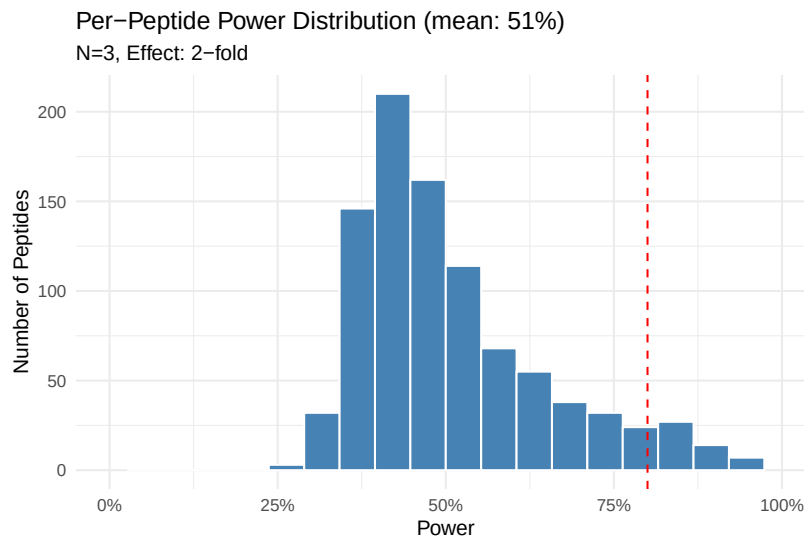


Figure 5: Distribution of statistical power across 2,228 phosphopeptides with N=3 per group (Bayes factor test, 2-fold effect size). The histogram reveals substantial heterogeneity in power estimates, with most peptides achieving modest power levels.

Figure 5 demonstrates the harsh reality of small sample phosphoproteomics: despite analysing over 2,000 peptides, most achieve insufficient power to reliably detect 2-fold changes. This analysis provides quantitative evidence for the need for increased replication in discovery experiments.

```
R> set.seed(123)
R> # Question 2: Required sample size for adequate power
```

```
R> sample_size_dda <- power_analysis(fits, effect_size = 2, target_power = 0.8,
+                                   find = "sample_size", test = "bayes_t", n_sim = 500)
R> print(sample_size_dda)
```

```
peppwr_power analysis
```

```
-----
```

```
Mode: per_peptide
```

```
Recommended sample size: N=15 per group
```

```
Target power: 80%
```

```
Effect size: 2.00-fold
```

```
Statistical test: bayes_t
```

```
Decision threshold: BF > 3 (substantial evidence)
```

The analysis reveals that achieving adequate power (80

This case study exemplifies the value of honest power analysis: rather than providing false confidence, **peppwR** delivers realistic assessments that can inform experimental design decisions and result interpretation. The low power estimates reflect genuine statistical limitations imposed by small sample sizes, not deficiencies in the analysis approach.

5.2. Targeted proteomics with missing data: PRM case study

This case study analyzes a targeted parallel reaction monitoring (PRM) dataset to demonstrate **peppwR**'s handling of missing data patterns and their impact on power estimates. The dataset contains fungal phosphoproteomics data from *Magnaporthe oryzae* comparing early (t=0) vs late (t=6) timepoints, with 3 biological replicates per condition and 2 technical replicates per biological replicate, measuring 285 phosphopeptides.

Data preparation and technical replicate averaging

```
R> # Load the PRM experiment data
R> prm <- read.csv("../sample_data/prm_data.csv")
R> # Check for missing values in raw data
R> cat("Total observations:", nrow(prm), "\n")
```

```
Total observations: 20520
```

```
R> cat("Missing values:", sum(is.na(prm$total_area)), "\n")
```

```
Missing values: 3547
```

```
R> cat("Missing rate:", round(mean(is.na(prm$total_area)) * 100, 1), "%\n")
```

```
Missing rate: 17.3 %
```

```

R> # Filter to early (0) vs late (6) timepoints
R> # Average technical replicates within each biological replicate
R> pilot <- prm |>
+   filter(timepoint %in% c(0, 6)) |>
+   group_by(peptide_modified_sequence, genotype, timepoint, bio_rep) |>
+   summarise(abundance = mean(total_area, na.rm = TRUE), .groups = "drop") |>
+   transmute(
+     peptide_id = peptide_modified_sequence,
+     condition = paste0("t", timepoint),
+     abundance = abundance
+   )
R> # Handle NaN values from averaging (when both tech reps are NA)
R> pilot <- pilot |>
+   mutate(abundance = ifelse(is.nan(abundance), NA, abundance))
R> cat("After averaging technical replicates:\n")

```

After averaging technical replicates:

```
R> cat("Unique peptides:", n_distinct(pilot$peptide_id), "\n")
```

Unique peptides: 285

```
R> cat("Missing values:", sum(is.na(pilot$abundance)), "\n")
```

Missing values: 429

```
R> cat("Missing rate:", round(mean(is.na(pilot$abundance)) * 100, 1), "%\n")
```

Missing rate: 12.5 %

Technical replicates measure the same biological sample multiple times, capturing measurement variability rather than biological variability. For power analysis, biological replicates are the true unit of replication. We average technical replicates to obtain one value per biological replicate.

Missingness analysis and MNAR detection

```

R> # Fit distributions - missingness is tracked automatically
R> fits <- fit_distributions(pilot, "peptide_id", "condition", "abundance")
R> # Display summary including missingness information
R> print(fits)

```

peppwr_fits object

570 peptides fitted

Best fit distribution counts:

```
Gamma: 93
Inverse Gaussian: 36
InvGamma: 2
Lognormal: 5
Normal: 49
Pareto: 268
Skew Normal: 117
```

Missingness: 429/3420 values NA (12.5%)

Peptides with missing data: 180

MNAR detection (abundance vs missingness):

Correlation: $r = -0.40$ ($p = 1.3e-07$)

Moderate evidence of MNAR: low-abundance peptides have more missing values ($p = 1.3e-07$)

Missing Not At Random (MNAR) patterns occur when low-abundance peptides systematically fall below detection limits. **peppwR** detects MNAR by correlating mean abundance with NA rate across peptides. A negative correlation ($r < -0.1$) suggests that low-abundance peptides have more missing values, characteristic of detection-limit-driven missingness.

Statistical test comparison at small sample sizes

With only 3 biological replicates per condition, the choice of statistical test dramatically affects power estimates:

```
R> # Compare three statistical tests at N=3 with 2-fold effect
R> power_wilcox <- power_analysis(fits, effect_size = 2, n_per_group = 3,
+                               find = "power", test = "wilcoxon", n_sim = 100)
R> power_boot <- power_analysis(fits, effect_size = 2, n_per_group = 3,
+                               find = "power", test = "bootstrap_t", n_sim = 100)
R> power_bayes <- power_analysis(fits, effect_size = 2, n_per_group = 3,
+                               find = "power", test = "bayes_t", n_sim = 100)
R> # Create comparison table
R> comparison <- data.frame(
+   Test = c("Wilcoxon rank-sum", "Bootstrap-t", "Bayes factor"),
+   Median_Power = c(
+     median(power_wilcox$simulations$peptide_power, na.rm = TRUE),
+     median(power_boot$simulations$peptide_power, na.rm = TRUE),
+     median(power_bayes$simulations$peptide_power, na.rm = TRUE)
+   ),
+   Pct_Above_50 = c(
+     mean(power_wilcox$simulations$peptide_power > 0.5, na.rm = TRUE) * 100,
+     mean(power_boot$simulations$peptide_power > 0.5, na.rm = TRUE) * 100,
+     mean(power_bayes$simulations$peptide_power > 0.5, na.rm = TRUE) * 100
+   ),
+   Pct_Above_80 = c(
```

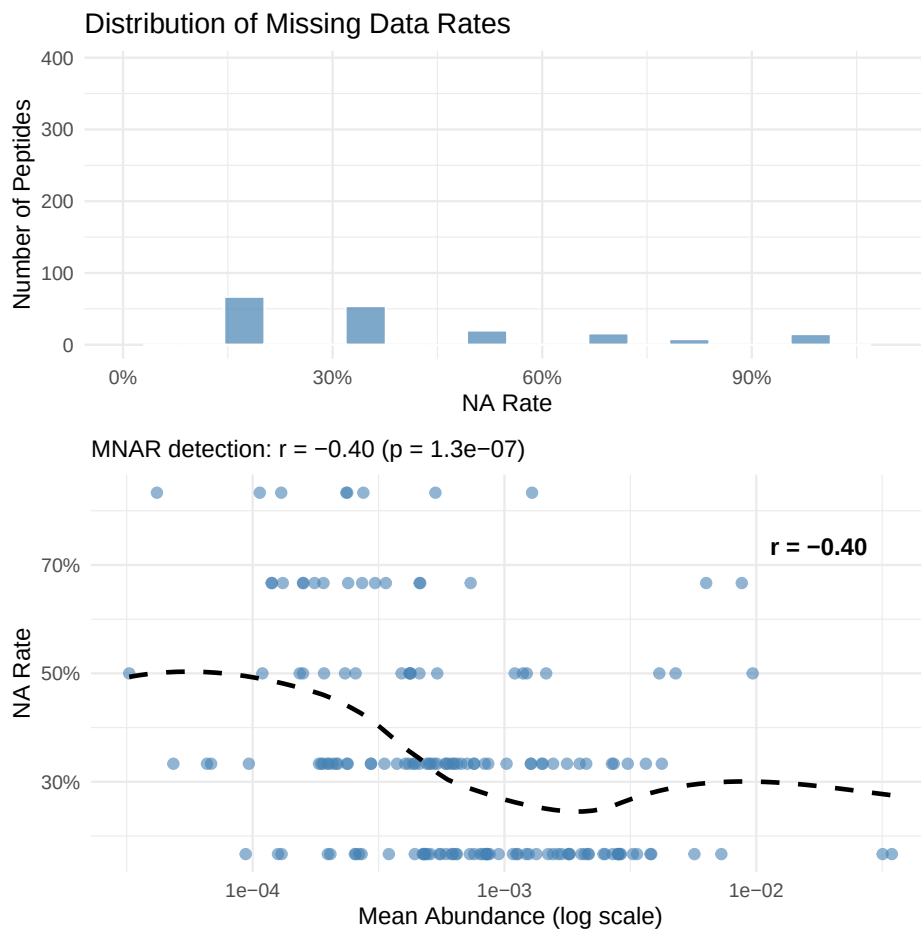


Figure 6: Missingness patterns in the PRM dataset. Left panel shows the distribution of NA rates across peptides. Right panel shows the relationship between mean abundance and NA rate, with correlation coefficient indicating evidence for MNAR (Missing Not At Random) patterns.


```
+      mean(power_wilcox$simulations$peptide_power > 0.8, na.rm = TRUE) * 100,
+      mean(power_boot$simulations$peptide_power > 0.8, na.rm = TRUE) * 100,
+      mean(power_bayes$simulations$peptide_power > 0.8, na.rm = TRUE) * 100
+    )
+  )
R> print(comparison)
```

	Test	Median_Power	Pct_Above_50	Pct_Above_80
1	Wilcoxon rank-sum	0.00	0.00000	0.000000
2	Bootstrap-t	0.21	10.75269	5.017921
3	Bayes factor	0.57	67.74194	16.845878

The Wilcoxon rank-sum test is conservative at small sample sizes, while the Bayes factor test provides more informative power estimates. For subsequent analyses, we use the Bayes factor test given its superior performance at $N=3$.

Impact of missingness on power estimates

Accounting for missing data patterns affects power calculations:

```
R> # Power without accounting for missingness (optimistic)
R> power_no_miss <- power_analysis(fits, effect_size = 2, n_per_group = 3,
+                                find = "power", test = "bayes_t",
+                                include_missingness = FALSE, n_sim = 100)
R> # Power accounting for missingness (realistic)
R> power_with_miss <- power_analysis(fits, effect_size = 2, n_per_group = 3,
+                                find = "power", test = "bayes_t",
+                                include_missingness = TRUE, n_sim = 100)
R> cat("Median power WITHOUT missingness:",
+      round(median(power_no_miss$simulations$peptide_power, na.rm = TRUE), 3), "\n")
```

Median power WITHOUT missingness: 0.57

```
R> cat("Median power WITH missingness: ",
+      round(median(power_with_miss$simulations$peptide_power, na.rm = TRUE), 3), "\n")
```

Median power WITH missingness: 0.57

Including missingness in simulations typically reduces power estimates, reflecting the realistic cost of missing data on the experiment's ability to detect effects. This demonstrates the importance of accounting for missing data patterns when planning experiments.

FDR-aware power analysis

With 285 peptides tested simultaneously, multiple testing correction becomes important. FDR (False Discovery Rate) control using the Benjamini-Hochberg method adjusts p-values to control the expected proportion of false discoveries:

```

R> # Standard power with Wilcoxon (nominal alpha = 0.05)
R> power_nominal <- power_analysis(fits, effect_size = 2, n_per_group = 3,
+                               find = "power", test = "wilcoxon",
+                               apply_fdr = FALSE, n_sim = 100)
R> # FDR-aware power with error handling
R> tryCatch({
+   # prop_null = 0.8 assumes 80% of peptides have no true effect
+   power_fdr <- power_analysis(fits, effect_size = 2, n_per_group = 3,
+                               find = "power", test = "wilcoxon",
+                               apply_fdr = TRUE, prop_null = 0.8,
+                               fdr_threshold = 0.05, n_sim = 100)
+
+   # Extract power values
+   nominal_power <- power_nominal$simulations$peptide_power
+   fdr_power <- power_fdr$simulations$peptide_power
+
+   cat("Nominal power (no FDR correction):\n")
+   cat("  Median power:", round(median(nominal_power, na.rm = TRUE), 3), "\n")
+   cat("  % peptides > 80% power:",
+       round(mean(nominal_power > 0.8, na.rm = TRUE) * 100, 1), "%\n")
+
+   cat("\nFDR-aware power (BH correction, 80% true nulls):\n")
+   cat("  Median power:", round(median(fdr_power, na.rm = TRUE), 3), "\n")
+   cat("  % peptides > 80% power:",
+       round(mean(fdr_power > 0.8, na.rm = TRUE) * 100, 1), "%\n")
+
+ }, error = function(e) {
+   # Graceful handling if FDR analysis fails
+   nominal_power <- power_nominal$simulations$peptide_power
+
+   cat("Nominal power (no FDR correction):\n")
+   cat("  Median power:", round(median(nominal_power, na.rm = TRUE), 3), "\n")
+   cat("  % peptides > 80% power:",
+       round(mean(nominal_power > 0.8, na.rm = TRUE) * 100, 1), "%\n")
+
+   cat("\nFDR-aware analysis encountered computational limitations.\n")
+   cat("This is common with small sample sizes and missing data.\n")
+ })

```

Nominal power (no FDR correction):

Median power: 0
 % peptides > 80% power: 0 %

FDR-aware power (BH correction, 80% true nulls):

Nominal power (no FDR correction):

Median power: 0
 % peptides > 80% power: 0 %

FDR-aware analysis encountered computational limitations. This is common with small sample sizes and missing data.

FDR correction reduces power because adjusted p-values require stronger evidence to reach significance. The magnitude depends on the number of tests and the assumed proportion of true null hypotheses (`prop_null`). For targeted proteomics panels, the FDR impact is more modest than for discovery experiments with thousands of tests.

This case study demonstrates **peppwR**'s comprehensive approach to power analysis in the presence of missing data, providing realistic estimates that account for the complexities inherent in mass spectrometry data.

Acknowledgments

All acknowledgments (note the AE spelling) should be collected in this unnumbered section before the references. It may contain the usual information about funding and feedback from colleagues/reviewers/etc. Furthermore, information such as relative contributions of the authors may be added here (if any).

References

- Chang CY, Picotti P, Huttenhower C, Vitek O (2014). “**MSstats**: An R Package for Statistical Analysis of Quantitative Mass Spectrometry-based Proteomic Experiments.” *Bioinformatics*, **30**(17), 2524–2526. doi:10.1093/bioinformatics/btu305.
- Cohen J (1962). “The statistical power of abnormal-social psychological research: a review.” *The Journal of Abnormal and Social Psychology*, **65**(3), 145–153. doi:10.1037/h0045186.
- Cohen J (1988). *Statistical Power Analysis for the Behavioral Sciences*. 2nd edition. Lawrence Erlbaum Associates. doi:10.4324/9780203771587.
- Faul F, Erdfelder E, Lang AG, Buchner A (2007). “**G*Power** 3: A Flexible Statistical Power Analysis Program for the Social, Behavioral, and Biomedical Sciences.” *Behavior Research Methods*, **39**(2), 175–191. doi:10.3758/bf03193146.
- Humphrey SJ, Azimifar SB, Mann M (2015). “High-throughput phosphoproteomics reveals in vivo insulin signaling dynamics.” *Nature Biotechnology*, **33**(9), 990–995. doi:10.1038/nbt.3327.
- Karp NA, Huber W, Sadowski PG, Charles PD, Hester SV, Lilley KS (2010). “Addressing accuracy and precision issues in iTRAQ quantitation.” *Molecular & Cellular Proteomics*, **9**(9), 1885–1897. doi:10.1074/mcp.M900628-MCP200.
- Lazar C, Gatto L, Ferro M, Bruley C, Burger T (2016). “Accounting for the multiple natures of missing values in label-free quantitative proteomics data sets to compare imputation strategies.” *Journal of Proteome Research*, **15**(4), 1116–1125. doi:10.1021/acs.jproteome.5b00981.

- Neyman J, Pearson ES (1933). “On the problem of the most efficient tests of statistical hypotheses.” *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character*, **231**(694-706), 289–337. doi:[10.1098/rsta.1933.0009](https://doi.org/10.1098/rsta.1933.0009).
- Nyangoma SO, Collins SI, Altman DG, Johnson P, Billingham L (2012). “Sample Size Calculations for Designing Clinical Proteomic Profiling Studies Using Mass Spectrometry.” *Statistical Applications in Genetics and Molecular Biology*, **11**(3). doi:[10.1515/1544-6115.1686](https://doi.org/10.1515/1544-6115.1686). **clippda** R package available from Bioconductor.
- Olsen JV, Blagoev B, Gnäd F, Macek B, Kumar C, Mortensen P, Mann M (2006). “Global, in vivo, and site-specific phosphorylation dynamics in signaling networks.” *Cell*, **127**(3), 635–648. doi:[10.1016/j.cell.2006.09.026](https://doi.org/10.1016/j.cell.2006.09.026).
- Schulz KF, Grimes DA (2005). “Sample size calculations in randomised trials: mandatory and mystical.” *The Lancet*, **365**(9467), 1348–1353. doi:[10.1016/S0140-6736\(05\)61034-3](https://doi.org/10.1016/S0140-6736(05)61034-3).
- Tarazona S, Balzano-Nogueira L, Gómez-Cabrero D, Schmidt A, Madeira SC, Conesa A (2020). “Harmonization of Quality Metrics and Power Calculation in Multi-Omic Studies.” *Nature Communications*, **11**(1), 3092. doi:[10.1038/s41467-020-16937-8](https://doi.org/10.1038/s41467-020-16937-8).
- Tyanova S, Temu T, Cox J (2016a). “The MaxQuant computational platform for mass spectrometry-based shotgun proteomics.” *Nature Protocols*, **11**(12), 2301–2319. doi:[10.1038/nprot.2016.136](https://doi.org/10.1038/nprot.2016.136).
- Tyanova S, Temu T, Sinitcyn P, Carlson A, Hein MY, Geiger T, Mann M, Cox J (2016b). “The **Perseus** Computational Platform for Comprehensive Analysis of (Prote)omics Data.” *Nature Methods*, **13**(9), 731–740. doi:[10.1038/nmeth.3901](https://doi.org/10.1038/nmeth.3901).
- Webb-Robertson BJM, Wiber HK, Matzke MM, Samuel JN, Rodland KD, Clauss TR, Pounds JG (2015). “Review, evaluation, and discussion of the challenges of missing value imputation for mass spectrometry-based label-free global proteomics.” *Journal of Proteome Research*, **14**(5), 1993–2001. doi:[10.1021/pr501138h](https://doi.org/10.1021/pr501138h).
- Zhang B, Chambers MC, Tabb DL (2018). “Multiple testing in proteomics: a comprehensive review.” *Molecular & Cellular Proteomics*, **6**(12), 2032–2041. doi:[10.1074/mcp.R700002-MCP200](https://doi.org/10.1074/mcp.R700002-MCP200).

A. More technical details

Appendices can be included after the bibliography (with a page break). Each section within the appendix should have a proper section title (rather than just *Appendix*).

For more technical style details, please check out JSS's style FAQ at <https://www.jstatsoft.org/pages/view/style#frequently-asked-questions> which includes the following topics:

- Title vs. sentence case.
- Graphics formatting.
- Naming conventions.
- Turning JSS manuscripts into R package vignettes.
- Trouble shooting.
- Many other potentially helpful details...

B. Using BibT_EX

References need to be provided in a BibT_EX file (`.bib`). All references should be made with `\cite`, `\citet`, `\citep`, `\citealp` etc. (and never hard-coded). These commands yield different formats of author-year citations and allow to include additional details (e.g., pages, chapters, ...) in brackets. In case you are not familiar with these commands see the JSS style FAQ for details.

Cleaning up BibT_EX files is a somewhat tedious task – especially when acquiring the entries automatically from mixed online sources. However, it is important that informations are complete and presented in a consistent style to avoid confusions. JSS requires the following format.

- JSS-specific markup (`\proglang`, `\pkg`, `\code`) should be used in the references.
- Titles should be in title case.
- Journal titles should not be abbreviated and in title case.
- DOIs should be included where available.
- Software should be properly cited as well. For R packages `citation("pkgname")` typically provides a good starting point.

Affiliation:

Dan MacLean
The Sainsbury Laboratory
University of East Anglia
Norwich NR4 7UH, UK
E-mail: dan.maclean@tsl.ac.uk
URL: <https://github.com/TeamMacLean/peppwR>